Universidad Mayor de San Andrés

Facultad de Ciencias Puras y Naturales

Carrera de Informática



Segundo Parcial - Escenario 3

Integrantes:

Bautista Villcacuti Cristian William	Nº 10	INF-381
Llujtaparo Machaca Tania	Nº 82	INF-381
Mamani Chavez Rafael Alejandro	Nº 92	INF-381
Mamani Quispe Dussan David	Nº 101	INF-381
Salgado Llusco Carlos Alejandro	Nº 133	INF-381

Docente: Lic. Rosa Flores Morales

Materia: Simulación de Sistemas

La Paz - Bolivia

2026

Escenario 3. Caja de supermercado en horario de baja demanda

Contexto: Una caja atiende a clientes que realizan compras pequeñas. El gerente considera que mantener la caja abierta durante las horas de baja demanda podría ser innecesario.

Disciplina: FIFO **Servidores:** 1

A. Tiempos entre llegadas

Tiempo (min)	Frecuencia
1	5
2	10
3	15
4	15
5	5
Total	50

B. Tiempos de servicio

Tiempo (min)	Frecuencia
1	10
2	20
3	15
4	5
Total	50

1. Análisis de datos, pruebas de ajuste y generación de variables aleatorias

1.1. Análisis estadístico de los datos

Se analizaron los datos proporcionados para los tiempos entre llegadas y los tiempos de servicio de los clientes, considerando un total de 50 observaciones para cada variable.

A. Tiempos entre llegadas

Tiempo (min)	Frecuencia	Frecuencia relativa
1	5	0.10
2	10	0.20
3	15	0.30
4	15	0.30
5	5	0.10

Total	**50**	1.00

Media:

$\bar{X} = \Sigma(xi \times fi) / N$

$\bar{X} = [(1\times5) + (2\times10) + (3\times15) + (4\times15) + (5\times5)] / 50$

$\bar{X} = 155 / 50 = 3{,}10$ minutos

Varianza:

$\sigma^2 = \Sigma[fi \times (xi - \bar{X})^2] / N$

$\sigma^2 = 64{,}50 / 50 = 1{,}29$ min²

Desviación estándar:

$\sigma = \sqrt{1{,}29} = 1{,}136$ minutos

Interpretación: Los clientes llegan en promedio cada 3.10 minutos. Los intervalos más frecuentes son de 3 y 4 minutos, representando cada uno el 30 % de las observaciones.

B. Tiempos de servicio

Tiempo (min)	Frecuencia	Frecuencia relativa
1	10	0.20
2	20	0.40
3	15	0.30
4	5	0.10
Total	**50**	**1.00**

Media:

$\bar{X} = \Sigma(xi \times fi) / N$

$\bar{X} = [(1\times10) + (2\times20) + (3\times15) + (4\times5)] / 50$

$\bar{X} = 115 / 50 =$ **2,30 minutos**

Varianza:

$\sigma^2 = \Sigma[fi \times (xi - \bar{X})^2] / N$

$\sigma^2 = 40{,}50 / 50 =$ **0,81 min²**

Desviación estándar:

$\sigma = \sqrt{0{,}81} =$ **0,90 minutos**

Interpretación: El cajero tarda en promedio 2.30 minutos en atender a cada cliente. El tiempo de servicio más frecuente es de 2 minutos, representando el 40 % de las observaciones.

1.2. Pruebas de ajuste

Para comprobar si los datos siguen una distribución uniforme discreta, se aplicó la prueba de bondad de ajuste chi-cuadrado, utilizando un nivel de significancia del 5 %.

Fórmula:

$\chi^2 = \Sigma[(Oi - Ei)^2 / Ei]$

Donde:

Oi = Frecuencia observada.

Ei = Frecuencia esperada.

A. Prueba para tiempos entre llegadas

Hipótesis:

H0: Los tiempos entre llegadas siguen una distribución uniforme discreta.

H1: Los tiempos entre llegadas no siguen una distribución uniforme discreta.

Frecuencia esperada:

Ei = 50 / 5 = 10

Tiempo	Observada	Esperada	Contribución
1	5	10	2.50
2	10	10	0.00

3	15	10	2.50
4	15	10	2.50
5	5	10	2.50
Total	**50**	**50**	**10.00**

χ^2 calculado = 10.00

Grados de libertad:

gl = 5 - 1 = 4

χ^2 crítico = 9.488

Como 10,00 es mayor que 9,488, se rechaza la hipótesis nula.

B. Prueba para tiempos de servicio

Hipótesis:

H0: Los tiempos de servicio siguen una distribución uniforme discreta.

H1: Los tiempos de servicio no siguen una distribución uniforme discreta.

Frecuencia esperada:

Ei = 50 / 4 = 12.50

Tiempo	**Observada**	**Esperada**	**Contribución**
1	10	12.50	0.50
2	20	12.50	4.50
3	15	12.50	0.50
4	5	12.50	4.50
Total	**50**	**50**	**10.00**

χ^2 calculado = 10,00

Grados de libertad:

gl = 4 - 1 = 3

χ^2 crítico = 7.815

Cómo 10.00 es mayor que 7.815, se rechaza la hipótesis nula.

Los resultados demuestran que ninguna de las dos variables sigue una distribución uniforme discreta. Por esta razón, se decidió utilizar distribuciones empíricas discretas, conservando las probabilidades observadas en los datos originales para realizar la simulación.

1.3. Generación de variables aleatorias

Construcción de las distribuciones acumuladas:

A. Distribución acumulada de los tiempos entre llegadas

Tiempo	Frecuencia	Probabilidad	Probabilidad acumulada
1	5	0.10	0.10
2	10	0.20	0.30
3	15	0.30	0.60
4	15	0.30	0.90
5	5	0.10	1.00

B. Distribución acumulada de los tiempos de servicio

Tiempo	Frecuencia	Probabilidad	Probabilidad acumulada
1	10	0.20	0.20
2	20	0.40	0.60
3	15	0.30	0.90
4	5	0.10	1.00

Aplicación de la transformada inversa:

A. Tiempos entre llegadas

Intervalo de U	Tiempo generado
$0 \leq U < 0.10$	1 minuto
$0.10 \leq U < 0.30$	2 minutos
$0.30 \leq U < 0.60$	3 minutos
$0.60 \leq U < 0.90$	4 minutos
$0.90 \leq U < 1.00$	5 minutos

B. Tiempos de servicio

Intervalo de U	Tiempo generado
$0 \leq U < 0.20$	1 minuto
$0.20 \leq U < 0.60$	2 minutos
$0.60 \leq U < 0.90$	3 minutos
$0.90 \leq U < 1.00$	4 minutos

Generación de números aleatorios uniformes entre 0 y 1:

El algoritmo para generar números aleatorios, verificar uniformidad e independencia se encuentra en la parte final del documento, el algoritmo que se utilizó fue el de generador congruencial aditivo. Para inicializar el generador se utiliza una semilla (en este caso se usó la semilla 101) para generar 10 números aleatorios de tiempos entre llegadas y 10 de tiempos de servicio.

Se obtuvieron los primeros cinco números aleatorios y se aplicó la transformada inversa para obtener los tiempos correspondientes.

N.º	U llegada	Tiempo entre llegadas	U servicio	Tiempo de servicio
1	0.5046	3	0.7127	3
2	0.6328	4	0.5385	2
3	0.0757	1	0.0006	1
4	0.5205	3	0.7961	3

5	0.1587	2	0.4502	2

Para reconstruir manualmente para 3 clientes consecutivos se utilizan los datos de la anterior tabla, Los tiempos entre llegadas obtenidos fueron de 3, 4 y 1 minuto, mientras que los tiempos de servicio fueron de 3, 2 y 1 minuto, respectivamente.

Cálculo del primer cliente. El primer cliente llega en el minuto 3. Como el servidor está libre, comienza su atención inmediatamente y finaliza en el minuto 6. Su tiempo de espera es de 0 minutos.

Cálculo del segundo cliente. El segundo cliente llega en el minuto 7, cuatro minutos después del primero. Como el servidor terminó su atención anterior en el minuto 6, comienza a ser atendido en el minuto 7 y finaliza en el minuto 9. Su tiempo de espera es de 0 minutos.

Cálculo del tercer cliente. El tercer cliente llega en el minuto 8. Sin embargo, el servidor se encuentra ocupado atendiendo al segundo cliente hasta el minuto 9. Por esta razón, comienza su atención en el minuto 9 y finaliza en el minuto 10, presentando un tiempo de espera de 1 minuto.

Resumen de la reconstrucción

Cliente	Hora de llegada	Inicio de servicio	Finalización	Tiempo de espera
1	3	3	6	0
2	7	7	9	0
3	8	9	10	1

Efectos de utilizar diferentes semillas aleatorias:

Al utilizar una semilla diferente se obtienen nuevas secuencias y, consecuentemente, pueden generarse distintos tiempos entre llegadas y tiempos de servicio.

Los resultados que permanecen aproximadamente estables al utilizar diferentes semillas aleatorias son las medias de los tiempos entre llegadas y de los tiempos de servicio. Esto se puede comprobar mediante la comparación de dos ejecuciones con semillas distintas. En la primera ejecución, utilizando la semilla 101, se obtuvo una media de tiempo entre llegadas de 2.8 y una

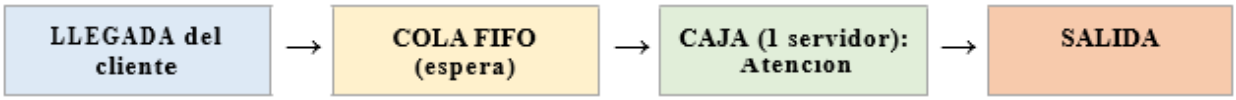
media de tiempo de servicio de 2.4. Por otro lado, en la segunda ejecución, con la semilla 573, se obtuvo una media de tiempo entre llegadas de 2.5 y una media de tiempo de servicio de 2.1.

Al comparar ambos resultados, se observa que los valores de las medias son relativamente cercanos, sin presentar diferencias significativas. Esto indica que, aunque la semilla aleatoria modifica los valores individuales generados en cada simulación, las medias tienden a mantenerse aproximadamente estables.

2. Algoritmo/configuración del modelo en Python

1. Diseño de la estructura del sistema de atención

El sistema es una cola de un solo servidor con disciplina FIFO. Cada cliente recorre el mismo camino: llega, espera si la caja está ocupada, es atendido y sale.



Elemento	Diseño
Entidad	Cliente
Recursos	1 servidor (caja) y una cola de capacidad ilimitada
Disciplina	FIFO: el primero en llegar es el primero en ser atendido
Eventos	LLEGADA y FIN de servicio (simulación por eventos discretos)
Estado inicial	Caja libre, cola vacía, reloj en 0
Unidad de tiempo	Minutos
Salida de cada réplica	Espera media y espera máxima de los 100 clientes

2. Configuración de la entidad Cliente

El cliente se representa con una clase que guarda los tiempos que se le asignan al generarlo (tiempo entre llegadas y tiempo de servicio) y las horas que se registran durante la simulación (llegada, inicio y fin). La espera no se guarda: se calcula como inicio − llegada.

```
        class Cliente:
            def __init__(self, ident, t_entre_llegadas, t_servicio):
                self.id = ident
self.t_entre = t_entre_llegadas    # tiempo desde el cliente anterior
                self.t_serv = t_servicio          # duracion de su atencion
                self.llegada = None
```

```
        self.inicio = None
        self.fin = None

    @property
    def espera(self):
        return self.inicio - self.llegada
```

3. Implementación de la cola FIFO

La cola es un collections.deque. Los clientes que encuentran la caja ocupada entran por el final con append(), y cuando el servidor se libera sale el primero con popleft(). Así se cumple FIFO sin necesidad de ordenar.

```
cola = deque()          # COLA FIFO (append al final, popleft al frente)
else:
                cola.append(cli)             # espera en la cola FIFO
if cola:
     iniciar_servicio(cola.popleft(), t)    # el primero de la cola pasa
```

Verificación: el programa guarda el orden en que los clientes inician servicio y comprueba con assert que sea creciente, por lo que cualquier violación de FIFO detiene la ejecución.

```
assert orden_servicio == sorted(orden_servicio), "se violo la disciplina
FIFO"
```

4. Configuración del servidor único

El servidor es una variable de estado, servidor_ocupado, que empieza en False (caja libre). La función iniciar_servicio() lo marca como ocupado, registra la hora de inicio del cliente y programa el evento de FIN de servicio en el momento inicio + tiempo de servicio.

```
servidor_ocupado = False       # estado inicial: servidor libre
def iniciar_servicio(cli, t):
        nonlocal servidor_ocupado
        servidor_ocupado = True
        cli.inicio = t
        orden_servicio.append(cli.id)
  heapq.heappush(eventos, (t + cli.t_serv, 1, cli.id, cli)) # evento FIN
```

Verificación: se comprueba que el inicio de cada cliente es posterior o igual al fin del anterior, es decir, que nunca hay dos clientes en la caja a la vez.

```
for a, b in zip(ordenados, ordenados[1:]):
assert b.inicio >= a.fin - 1e-9, "dos clientes en el servidor a la vez"
```

5. Programación de los tiempos entre llegadas y de servicio

Los tiempos se generan con la transformada inversa sobre la distribución acumulada empírica de los 50 datos observados: se genera $U \sim U(0,1)$ y se toma el primer valor x tal que $U \leq F(x)$.

Variable	Valor (min)	F(x)	Intervalo de U
Entre llegadas	1	0.10	$0.00 < U \leq 0.10$
Entre llegadas	2	0.30	$0.10 < U \leq 0.30$
Entre llegadas	3	0.60	$0.30 < U \leq 0.60$
Entre llegadas	4	0.90	$0.60 < U \leq 0.90$
Entre llegadas	5	1.00	$0.90 < U \leq 1.00$
Servicio	1	0.20	$0.00 < U \leq 0.20$
Servicio	2	0.60	$0.20 < U \leq 0.60$

Variable	Valor (min)	F(x)	Intervalo de U
Servicio	3	0.90	$0.60 < U \leq 0.90$
Servicio	4	1.00	$0.90 < U \leq 1.00$

```
N_CLIENTES = 100          # clientes por replica
SERVIDORES = 1            # servidor unico
LLEGADAS = {1: 5, 2: 10, 3: 15, 4: 15, 5: 5}
SERVICIO = {1: 10, 2: 20, 3: 15, 4: 5}
def acumulada(dic):
    """Devuelve (valores, F(x)) de la distribucion empirica."""
    total = sum(dic.values())
    xs = sorted(dic)
    F, acum = [], 0.0
    for x in xs:
        acum += dic[x] / total
        F.append(acum)
    return xs, F


XS_LLEG, F_LLEG = acumulada(LLEGADAS)
```

```
XS_SERV, F_SERV = acumulada(SERVICIO)


def inversa(u, xs, F):
    """Transformada inversa discreta: primer x con u <= F(x)."""
    for x, f in zip(xs, F):
        if u <= f + 1e-12:
            return x
    return xs[-1]
```

Cada cliente recibe sus dos tiempos al crearse, con dos números aleatorios independientes (uno para llegada y otro para servicio):

```
rng = np.random.default_rng(semilla)

    def nuevo_cliente(i):
        ua, us = rng.random(), rng.random()       # numeros U(0,1)
return Cliente(i,inversa(ua,XS_LLEG,F_LLEG), inversa(us,XS_SERV, F_SERV))
```

6. Lógica de llegada, espera, atención y salida

El reloj avanza de evento en evento. Los eventos futuros se guardan en un heap ordenado por tiempo, y en cada paso se procesa el más próximo.

Evento	Qué ocurre
LLEGADA	Se programa la llegada del siguiente cliente (hasta completar 100). Si la caja está libre, el cliente pasa a atención con espera 0; si no, se une a la cola FIFO.
FIN de servicio	El cliente sale del sistema (se registra su hora de fin). La caja queda libre y, si hay cola, entra a atención el primer cliente de la fila.

```
cola = deque()            # COLA FIFO (append al final, popleft al frente)

    servidor_ocupado = False       # estado inicial: servidor libre

    eventos = []              # lista de eventos futuros (heap por tiempo)

    atendidos = []                 # clientes que ya salieron

    orden_servicio = []            # para verificar FIFO
```

```
    def iniciar_servicio(cli, t):
        nonlocal servidor_ocupado
        servidor_ocupado = True
        cli.inicio = t
        orden_servicio.append(cli.id)
  heapq.heappush(eventos, (t + cli.t_serv, 1, cli.id, cli))   # evento FIN
# primera llegada: reloj 0 + primer tiempo entre llegadas
    c = nuevo_cliente(1)
    c.llegada = c.t_entre
    heapq.heappush(eventos, (c.llegada, 0, c.id, c))    # evento LLEGADA
    generados = 1

    while eventos:
        t, tipo, _, cli = heapq.heappop(eventos)
        if tipo == 0:                              # ---- LLEGADA ----
            if generados < n_clientes:     # programa la siguiente llegada
                generados += 1
                sig = nuevo_cliente(generados)
                sig.llegada = t + sig.t_entre
                heapq.heappush(eventos, (sig.llegada, 0, sig.id, sig))
            if not servidor_ocupado:
                iniciar_servicio(cli, t)   # atencion inmediata (espera 0)
            else:
```

```
            cola.append(cli)                    # espera en la cola FIFO
        else:                                  # ---- FIN DE SERVICIO ----
            cli.fin = t
            atendidos.append(cli)                          # SALIDA del sistema
            servidor_ocupado = False
            if cola:
    iniciar_servicio(cola.popleft(), t)      # el primero de la cola pasa
```

7. Garantía de 100 clientes por réplica

La réplica genera exactamente N_CLIENTES = 100 llegadas: un contador (generados) detiene la creación de nuevos clientes al llegar a 100, y la simulación termina cuando se vacía la lista de eventos, es decir, cuando los 100 ya salieron. Al final se comprueba con assert:

```
assert len(atendidos) == n_clientes, "la replica no proceso 100 clientes"
```

Evidencia: se ejecutaron 124 réplicas (semillas 1000 a 1123) y en todas salieron exactamente 100 clientes, sin que se activara ninguna aserción. Además, en la ejecución de 5 réplicas la columna Clientes muestra 100 en cada una.

8. Ejecución del modelo

Para ejecutarlo: instalar numpy y scipy (pip install numpy scipy) y correr python modelo_caja.py, o pegar el código en una celda de Google Colab. Salida obtenida:

```
 Rep  Semilla  Clientes   Espera media  Espera max
  1     101      100         0.700         4.0
  2     202      100         0.590         5.0
  3     303      100         0.260         3.0
  4     404      100         0.600         5.0
  5     505      100         0.400         3.0

Media=0.510  s=0.177  IC95%=[0.290; 0.730]  error
relativo=43.1%
```

```
Traza de los primeros 5 clientes (semilla 101):
 Cli   T.entre Llegada  Inicio  T.serv  Fin   Espera
  1       5      5.0     5.0       2    7.0     0.0
  2       4      9.0     9.0       2   11.0     0.0
  3       2     11.0    11.0       4   15.0     0.0
  4       4     15.0    15.0       2   17.0     0.0
  5       5     20.0    20.0       2   22.0     0.0
```

En la traza se ve la lógica del modelo: cada cliente inicia su servicio en max(llegada, fin del anterior) y su espera es inicio − llegada. En la réplica de semilla 101, 43 de los 100 clientes esperaron algo en la cola.

3. Tabla con resultados de las réplicas

Considerando las 5 réplicas, la tabla es la siguiente:

Réplica	Semilla	Clientes procesados	Tiempo medio de espera en cola Xi	Tiempo máximo de espera
1	101	100	**0.6200 min**	5.0 min
2	202	100	**0.5400 min**	5.0 min
3	303	100	**0.7800 min**	5.0 min
4	404	100	**0.5100 min**	4.0 min
5	505	100	**0.4000 min**	3.0 min

Cálculo inferencial

A. Media

$$\overline{X} = \frac{\sum_{i=1}^{5} \overline{X}_i}{5} = \frac{0.62+0.54+0.78+0.51+0.40}{5} = 0.57\, min$$

B. Desviación estándar

$$S = \sqrt{\frac{\Sigma\left(\overline{X}_i - \overline{X}\right)^2}{5-1}} = \sqrt{\frac{(0.62-0.57)^2+(0.54-0.57)^2+(0.78-0.57)^2+(0.51-0.57)^2+(0.4-0.57)^2}{4}}$$

$$S = \sqrt{\frac{0.0025+0.0009+0.0441+0.0036+0.0289}{4}} = 0.1414\, min$$

C. Intervalo de confianza al 95%

$$t_{\alpha/2,\, n-1} = t_{0.025,\, 4} = 2.7764$$

- **Margen de error**

$$h = t_{0.025,\, 4} \cdot \frac{S}{\sqrt{n}} = 2.7764 \cdot \frac{0.1414}{\sqrt{5}} = 0.1756\, min$$

- **Intervalo de confianza**

$$IC_{95\%} = [\overline{X} - h,\ \overline{X} + h] = [0.57 - 0.1756,\ 0.57 + 0.1756] = [0.3944;\ 0.7456] min$$

D. Evaluación

- **Error relativo (Er)**

$$Er = \frac{h}{\overline{X}} = \frac{0.1756}{0.51} = 0.3081 \Rightarrow 30.81\%$$

El criterio exige un **error relativo máximo del 10%** ($\epsilon = 0.10$). Al ser 43.07% > 10%, las 5 réplicas iniciales no alcanzan la precisión requerida.

- **Cálculo del número de réplicas necesarias (r)**

Usando la fórmula:

$$r_{adicionales} = r_{actuales}\left(\frac{Er}{e}\right)^2 = 5 \cdot \left(\frac{0.3081}{0.10}\right)^2 = 47.45$$

$$r_{adicionales} = 48\,réplicas$$

$$Réplicas\ adicionales\ requeridas = 48 - 5 = 43\,réplicas$$

Resultados globales con réplicas adicionales n = 89

$Media$ $\quad \overline{X} = 0.5996\,min$

$Desviación\ Estándar$ $\quad S = 0.2838\,min$

$Margen\ de\ Error$ $\quad h = 0.1756\,min$

$Intervalo\ de\ Confianza$ $\quad IC_{95\%} = [0.5398;\ 0.6593]min$

$Error\ Relativo$ $\quad Er = 0.0997 \Rightarrow 9.97\% \leq 10\%\ \ Cumple$

4. CONCLUSIONES Y RESPUESTAS CRÍTICAS

4.1. Preguntas técnicas de verificación

1. Presenten los primeros cinco números aleatorios utilizados y muestren cómo transformaron al menos dos de ellos en tiempos entre llegadas y tiempos de servicio.

Respuesta:

Para generar los tiempos entre llegadas y los tiempos de servicio se utilizó un generador congruencial aditivo, inicializado con la semilla 101.

Se generaron dos secuencias pseudoaleatorias independientes: una para los tiempos entre llegadas y otra para los tiempos de servicio. Los números obtenidos se encuentran entre 0 y 1 y se transformaron mediante el método de la transformada inversa, utilizando las distribuciones empíricas acumuladas de los datos observados.

Los primeros cinco números aleatorios obtenidos fueron:

N.º	U llegada	T. entre llegadas	U servicio	T. servicio
1	0,5046	3 min	0,7127	3 min
2	0,6328	4 min	0,5385	2 min
3	0,0757	1 min	0,0006	1 min
4	0,5205	3 min	0,7961	3 min
5	0,1587	2 min	0,4502	2 min

Ejemplo de transformación:

Para los tiempos entre llegadas, la distribución acumulada es:

- 1 minuto: 0,10

- 2 minutos: 0,30
- 3 minutos: 0,60
- 4 minutos: 0,90
- 5 minutos: 1,00

Si obtenemos el número aleatorio 0,5046, observamos que se encuentra entre 0,30 y 0,60. Por lo tanto, el tiempo entre llegadas correspondiente es de 3 minutos.

Para los tiempos de servicio, la distribución acumulada es:

- 1 minuto: 0,20
- 2 minutos: 0,60
- 3 minutos: 0,90
- 4 minutos: 1,00

Si obtenemos el número aleatorio 0,7127, observamos que se encuentra entre 0,60 y 0,90. Por lo tanto, el tiempo de servicio correspondiente es de 3 minutos.

De esta manera, los números pseudoaleatorios uniformes entre 0 y 1 se transforman en tiempos discretos que respetan las probabilidades obtenidas de las observaciones originales.

2. Seleccionen tres clientes consecutivos y reconstruyan manualmente su hora de llegada, inicio de servicio, finalización y tiempo de espera.

Respuesta:

Se seleccionaron los tres primeros clientes de la réplica con semilla 101.

- **Cliente 1:** su tiempo entre llegadas es de 3 minutos, por lo que llega en el minuto 3. Como el servidor está libre, inicia su atención inmediatamente y termina en el minuto 6. Su tiempo de espera es de 0 minutos.
- **Cliente 2:** su tiempo entre llegadas es de 4 minutos, por lo que llega en el minuto 7. El servidor terminó de atender al primer cliente en el minuto 6, por lo que

comienza su atención inmediatamente y finaliza en el minuto 9. Su tiempo de espera es de 0 minutos.

- **Cliente 3:** su tiempo entre llegadas es de 1 minuto, por lo que llega en el minuto 8. Sin embargo, el servidor todavía está atendiendo al segundo cliente hasta el minuto 9. Por esta razón, debe esperar hasta que el servidor quede libre. Inicia su atención en el minuto 9 y finaliza en el minuto 10. Su tiempo de espera es de 1 minuto.

Resumen de la reconstrucción:

Cliente	Llegada	Inicio de servicio	Finalización	Espera
1	3	3	6	0
2	7	7	9	0
3	8	9	10	1

Los resultados coinciden con los obtenidos por el programa, comprobando que el cálculo de las llegadas, los inicios de atención, las finalizaciones y los tiempos de espera funciona correctamente.

3. ¿Qué resultados cambian al utilizar otra semilla aleatoria y cuáles deberían permanecer aproximadamente estables?

Respuesta:

Al utilizar una semilla aleatoria diferente se generan nuevas secuencias de números pseudoaleatorios. Por lo tanto, cambian los tiempos individuales entre llegadas, los tiempos de servicio, los tiempos de espera de cada cliente y los resultados particulares de cada réplica.

Sin embargo, deberían permanecer aproximadamente estables los indicadores generales cuando se realizan suficientes réplicas, como el tiempo medio de espera, la utilización del servidor y las proporciones de clientes que esperan o son atendidos inmediatamente.

Esto ocurre porque todas las réplicas utilizan las mismas distribuciones empíricas y las mismas condiciones del sistema, aunque los números aleatorios generados sean diferentes.

4. Expliquen cómo obtuvieron el intervalo de confianza y qué significa para el tiempo medio de espera.

Respuesta:

El intervalo de confianza se calculó utilizando la media y la desviación estándar de los resultados de las 89 réplicas, junto con el valor crítico de la distribución t de Student para un nivel de confianza del 95 %.

Los resultados finales fueron:

- Número de réplicas: 89.
- Media de espera: 0,5996 minutos.
- Desviación estándar: 0,2838 minutos.
- Margen de error: 0,0598 minutos.

El intervalo obtenido fue:

IC 95 % = [0,5398; 0,6593] minutos.

Esto significa que, mediante el procedimiento estadístico utilizado, se obtiene un intervalo de confianza del 95 % para estimar el verdadero tiempo medio de espera del sistema.

En términos prácticos, el tiempo medio de espera estimado es de aproximadamente 0,60 minutos, es decir, cerca de 36 segundos.

Además, se obtuvo un error relativo del 9,97 %, cumpliendo con el criterio establecido de no superar el 10 %.

5. Si duplicaran el número de réplicas, ¿qué esperan que suceda con la amplitud del intervalo de confianza?

Respuesta:

Si se duplica el número de réplicas, se espera que la amplitud del intervalo de confianza disminuya, siempre que la variabilidad de los resultados se mantenga aproximadamente estable.

Esto sucede porque al aumentar el número de réplicas disminuye el error estándar de la media, lo que permite obtener una estimación más precisa del tiempo medio de espera.

Sin embargo, duplicar el número de réplicas no significa que el intervalo se reduzca exactamente a la mitad. Su amplitud disminuye aproximadamente en proporción inversa a la raíz cuadrada del número de réplicas.

4.2 Preguntas de pensamiento crítico:

1. Si el modelo produce una cola creciente, ¿cómo distinguirían entre un error de programación y una demanda superior a la capacidad del servidor?

Respuesta:

Primero verificaríamos que el programa funcione correctamente, comprobando la disciplina FIFO, el procesamiento de los eventos, los tiempos generados y que el servidor atienda a un solo cliente a la vez.

Después compararíamos la frecuencia de llegada de los clientes con la capacidad de atención del servidor.

Si los clientes llegan más rápido de lo que el servidor puede atenderlos, la cola puede crecer debido a que la demanda supera la capacidad de atención.

En cambio, si existe una inconsistencia en la programación, como errores en los tiempos o en la asignación de eventos, la cola podría crecer incluso cuando la capacidad del servidor sea suficiente.

En nuestro escenario, el tiempo medio entre llegadas es de 3,10 minutos y el tiempo medio de servicio es de 2,30 minutos. Esto indica que, en promedio, el servidor tiene capacidad suficiente para atender la demanda durante el período analizado.

2. ¿Puede un modelo tener un intervalo de confianza muy estrecho y, sin embargo, representar mal la realidad?

Respuesta:

Sí. Un intervalo de confianza estrecho indica que los resultados de las simulaciones presentan poca incertidumbre estadística respecto a la media estimada.

Sin embargo, esto no garantiza que el modelo represente correctamente la realidad.

Por ejemplo, si los tiempos de llegada o de servicio no representan adecuadamente el comportamiento real de los clientes, los resultados podrían ser estadísticamente precisos, pero incorrectos en la práctica.

Por eso es importante verificar tanto el funcionamiento del programa como la calidad de los datos y las condiciones utilizadas para construir el modelo.

3. Si una réplica da un resultado favorable y otra desfavorable, ¿cuál presentarían a la gerencia? ¿Por qué?

Respuesta:

Presentaríamos todas las réplicas y sus resultados, incluyendo los favorables y desfavorables, junto con la media general, la desviación estándar y el intervalo de confianza.

No sería correcto seleccionar únicamente la réplica que presente el resultado más conveniente.

La gerencia necesita conocer el comportamiento general del sistema y también las situaciones desfavorables que podrían afectar a los clientes.

De esta manera, las decisiones se basarían en el conjunto de los resultados y no en un caso aislado.

4. ¿Qué cambiarían primero para reducir la espera: el tiempo de servicio, la política de atención o el número de servidores? ¿Qué evidencia necesitarían?

Respuesta:

Primero analizaríamos la posibilidad de reducir el tiempo de servicio mediante mejoras en el proceso de atención, siempre que no se afecte la calidad del servicio.

También podríamos evaluar otros escenarios, como modificar la política de atención o incorporar un servidor adicional.

Para tomar una decisión necesitaríamos comparar diferentes alternativas mediante nuevas simulaciones, utilizando las mismas condiciones de demanda y suficientes réplicas.

Además, analizaríamos el tiempo medio de espera, el tiempo máximo, la utilización de los servidores y los costos de cada alternativa.

5. Si la media de espera disminuye, pero aumenta el tiempo máximo de espera, ¿considerarían que la modificación fue exitosa?

Respuesta:

No necesariamente.

Aunque una disminución del tiempo medio de espera representa una mejora para la mayoría de los clientes, un aumento del tiempo máximo puede indicar que algunos clientes están experimentando esperas considerablemente mayores.

Por lo tanto, evaluaríamos ambos indicadores antes de considerar exitosa la modificación.

También sería importante analizar cuántos clientes se ven afectados por esas esperas prolongadas y si la mejora general compensa los casos desfavorables.

4.3 Preguntas del escenario:

1. ¿Un tiempo promedio de espera cercano a cero justifica cerrar la caja?

Respuesta:

No necesariamente.

En nuestro modelo, el tiempo medio de espera fue de 0,5996 minutos, equivalente aproximadamente a 36 segundos, lo que representa una espera relativamente reducida.

Sin embargo, un tiempo medio de espera bajo no significa que ningún cliente tenga que esperar. Algunos clientes pueden encontrar la caja ocupada y necesitar esperar antes de recibir atención.

Además, la caja cumple una función importante al permitir que los clientes sean atendidos durante el horario de baja demanda.

Por lo tanto, no recomendaríamos cerrar la caja únicamente porque el tiempo promedio de espera sea bajo. Antes de tomar esa decisión sería necesario analizar los costos de mantenerla abierta y las consecuencias que tendría para los clientes.

2. ¿Qué costos o beneficios no están representados en el modelo?

Respuesta:

El modelo se concentra principalmente en los tiempos de llegada, servicio y espera de los clientes. Sin embargo, existen otros factores económicos y operativos que no están incluidos.

Costos no representados:

- Salario del cajero.
- Consumo de electricidad.
- Mantenimiento y funcionamiento de la caja.
- Costos de equipos y recursos utilizados.

- Costos administrativos y operativos.

Beneficios no representados:

- Mayor satisfacción de los clientes.
- Reducción de la incomodidad y las molestias durante la espera.
- Mejor atención en períodos de mayor demanda.
- Fidelización de los clientes.
- Prevención de pérdidas de ventas por clientes que decidan retirarse al encontrar una fila.

Por esta razón, los resultados de la simulación deben complementarse con un análisis económico y operativo antes de decidir si conviene mantener abierta o cerrar la caja.

3. ¿Cómo distinguirían una mejora real de una diferencia debida al azar?

Respuesta:

Para distinguir una mejora real de una diferencia debida al azar, compararíamos el sistema original con el sistema modificado mediante varias réplicas independientes.

Ambas alternativas deberían utilizar las mismas condiciones de demanda y los mismos criterios de medición.

Después analizaríamos las medias de espera, los intervalos de confianza y otros indicadores importantes.

Si la mejora se mantiene de forma consistente en las diferentes réplicas y existe evidencia estadística suficiente de que la reducción no se debe únicamente a la variabilidad aleatoria, podríamos considerar que se trata de una mejora real.

4.4 Conclusiones finales

1. Se comprobó que el modelo de simulación representa el funcionamiento de una caja de supermercado con un único servidor y una cola FIFO. Las verificaciones

realizadas permiten comprobar que se respeta el orden de atención y que no se atiende a dos clientes simultáneamente.

2. La reconstrucción manual de los primeros tres clientes de la réplica con semilla 101 permitió verificar que los tiempos de llegada, inicio de atención, finalización y espera coinciden con los resultados generados por el programa.
3. Los tiempos generados mediante el generador congruencial aditivo y la transformada inversa utilizan como referencia las distribuciones empíricas obtenidas de las 50 observaciones de los tiempos entre llegadas y de servicio.
4. Se ejecutaron inicialmente cinco réplicas, con las cuales se obtuvo un error relativo del 30,81 %. Mediante la fórmula de estimación se calcularon 48 réplicas teóricas. Sin embargo, al continuar agregando ejecuciones y recalculando el error relativo, el programa alcanzó un total de 89 réplicas.
5. Después de las 89 réplicas se obtuvo un tiempo medio de espera de 0,5996 minutos, con un intervalo de confianza del 95 % entre 0,5398 y 0,6593 minutos. El error relativo fue del 9,97 %, cumpliendo con el criterio de precisión establecido del 10 %.
6. Los resultados muestran que, durante el horario de baja demanda analizado, el sistema presenta un tiempo medio de espera relativamente reducido. Sin embargo, esto no significa que todos los clientes sean atendidos inmediatamente, ya que pueden producirse situaciones de espera cuando el servidor se encuentra ocupado.
7. Finalmente, no se recomienda cerrar la caja únicamente porque el tiempo medio de espera sea bajo. Es necesario considerar también los costos de funcionamiento, la satisfacción de los clientes y las posibles consecuencias de reducir la capacidad de atención. Por lo tanto, la decisión debe basarse en una evaluación conjunta de los resultados operativos y económicos.

Conclusión general: el modelo permitió evaluar el funcionamiento de la caja del supermercado, comprobar su comportamiento mediante simulaciones y obtener resultados estadísticamente aceptables. La decisión de mantener abierta o cerrar la caja no debe basarse exclusivamente en el tiempo medio de espera, sino también en la utilización del servidor, los tiempos máximos, la demanda y los factores económicos que no fueron incluidos en el modelo.

5. Captura o demostración del modelo en ejecución

Código de algoritmo para generar números aleatorios con python:

```
import math, bisect
from scipy.stats import chi2, norm

class CongruencialAditivo:
    def __init__(self, semilla, j=24, k=55, m=2**31):
        self.j, self.k, self.m = j, k, m
        x, self.hist = semilla, []
        for _ in range(k):
            x = (1103515245 * x + 12345) % 2**31
            self.hist.append(x)

    def siguiente(self):
        x = (self.hist[-self.j] + self.hist[-self.k]) % self.m
        self.hist.append(x); self.hist.pop(0)
        return x

    def random(self):
        return self.siguiente() / self.m

llegadas = {1: 0.10, 2: 0.20, 3: 0.30, 4: 0.30, 5: 0.10}
servicio = {1: 0.20, 2: 0.40, 3: 0.30, 4: 0.10}

def construir_cdf(dist):
    valores, acum, s = [], [], 0
    for v, p in dist.items():
        s += p
        valores.append(v); acum.append(round(s, 10))
    return valores, acum

def inversa(u, valores, acum):
    return valores[bisect.bisect_right(acum, u)]

cdf_a, cdf_s = construir_cdf(llegadas), construir_cdf(servicio)

def generar(n, semilla):
    ga = CongruencialAditivo(semilla)
    gs = CongruencialAditivo(semilla + 1000)
    ua = [ga.random() for _ in range(n)]
    us = [gs.random() for _ in range(n)]
    ta = [inversa(u, *cdf_a) for u in ua]
    ts = [inversa(u, *cdf_s) for u in us]
    return ua, ta, us, ts

def prueba_media(u, alfa=0.05):
    n = len(u); xbar = sum(u) / n
    z0 = (xbar - 0.5) * math.sqrt(n) / math.sqrt(1/12)
    zc = norm.ppf(1 - alfa/2)
    li = 0.5 - zc / math.sqrt(12 * n)
    ls = 0.5 + zc / math.sqrt(12 * n)
    return {"media": xbar, "Z0": z0, "Zcrit": zc, "lim_inf": li, "lim_sup": ls,
"acepta": li <= xbar <= ls}
```

```
def prueba_varianza(u, alfa=0.05):
    n = len(u); xbar = sum(u) / n
    s2 = sum((x - xbar)**2 for x in u) / (n - 1)
    li = chi2.ppf(alfa/2, n - 1) / (12 * (n - 1))
    ls = chi2.ppf(1 - alfa/2, n - 1) / (12 * (n - 1))
    return {"varianza": s2, "lim_inf": li, "lim_sup": ls, "acepta": li <= s2 <=
ls}

def prueba_corridas(u, alfa=0.05):
    n = len(u)
    s = [1 if u[i+1] > u[i] else 0 for i in range(n - 1)]
    c0 = 1 + sum(1 for i in range(1, len(s)) if s[i] != s[i-1])
    mu = (2*n - 1) / 3
    var = (16*n - 29) / 90
    z0 = (c0 - mu) / math.sqrt(var)
    zc = norm.ppf(1 - alfa/2)
    return {"C0": c0, "mu_C": mu, "var_C": var, "Z0": z0, "Zcrit": zc, "acepta":
abs(z0) < zc}

def decision(p):
    return "No se rechaza" if p["acepta"] else "SE RECHAZA"

def reporte_pruebas(nombre, u, alfa=0.05):
    pm, pv, pc = prueba_media(u, alfa), prueba_varianza(u, alfa),
prueba_corridas(u, alfa)
    print(f"\n___Flujo: {nombre} (n = {len(u)}, alfa = {alfa})___")
    print(f"{'Prueba':<22}{'Estadístico':<28}{'Criterio de
aceptación':<30}{'Decisión'}")
    print("-" * 95)
    print(f"{'Media':<22}{'media = %.4f' % pm['media']:<28}"
          f"{'%.4f <= media <= %.4f' % (pm['lim_inf'],
pm['lim_sup']):<30}{decision(pm)}")
    print(f"{'Varianza':<22}{'s2 = %.4f' % pv['varianza']:<28}"
          f"{'%.4f <= s2 <= %.4f' % (pv['lim_inf'],
pv['lim_sup']):<30}{decision(pv)}")
    print(f"{'Corridas arriba/abajo':<22}{'C_0 = %d, Z_0 = %.3f' % (pc['C0'],
pc['Z0']):<28}"
          f"{'|Z_0| < %.3f' % pc['Zcrit']:<30}{decision(pc)}")

if __name__ == "__main__":
    n = 10
    semilla = 101

    ua, ta, us, ts = generar(n, semilla)

    print(f"SEMILLA: {semilla}   |   n = {n}")
    print(f"\n___Primeros {n*2} números aleatorios y su transformación___")
    print(f"{'#':<4}{'U llegada':<12}{'T. entre llegadas':<20}{'U
servicio':<12}{'T. servicio'}")
    for i in range(n):
        print(f"{i+1:<4}{ua[i]:<12.4f}{ta[i]:<20}{us[i]:<12.4f}{ts[i]}")

    print("\n___Medias generadas___")
    print(f"Entre llegadas: {sum(ta)/n:.2f}")
    print(f"Servicio:       {sum(ts)/n:.2f}")
    reporte_pruebas("llegadas", ua)
```

```
reporte_pruebas("servicio", us)
```

```
TABLA 1: REPLICAS INICIALES (n = 5)
 Rep |  Semilla |  Clientes |   Espera Media (min) |  Espera Max
-------------------------------------------------------------
   1 |      101 |       100 |               0.6200 |         5.0
   2 |      202 |       100 |               0.5400 |         5.0
   3 |      303 |       100 |               0.7800 |         5.0
   4 |      404 |       100 |               0.5100 |         4.0
   5 |      505 |       100 |               0.4000 |         3.0
-------------------------------------------------------------
Media global (X_bar):    0.5700 min
Desviacion estandar (s): 0.1414 min
Margen de error (h):     0.1756 min
Intervalo 95%:           [0.3944; 0.7456] min
Error relativo actual:   30.81%

>> Calculo segun formula de ProModel: n* = 5 * (0.3081 / 0.10)^2 = 47.45
>> Replicas totales estimadas requeridas: 48
>> Replicas adicionales necesarias:       43

=====================================================================
TABLA 2: RESULTADOS GLOBALES CON REPLICAS ADICIONALES (n = 89)
=====================================================================
Replicas totales ejecutadas:    89
Media final de espera:          0.5996 min
Desviacion estandar final:      0.2838 min
Margen de error final (h):      0.0598 min
Intervalo de Confianza al 95%:  [0.5398; 0.6593] min
Error relativo alcanzado:       9.97% (<= 10.00% CUMPLE)

VERIFICACION: Reconstruccion de los primeros 3 clientes (Semilla 101):
 Cli |  T.entre |  Llegada |   Inicio |   T.serv |    Fin |  Espera
-----------------------------------------------------------------
   1 |        3 |      3.0 |      3.0 |        3 |    6.0 |     0.0
   2 |        4 |      7.0 |      7.0 |        2 |    9.0 |     0.0
   3 |        1 |      8.0 |      9.0 |        1 |   10.0 |     1.0
```